

Assignment: Multi-Tenant Agentic WhatsApp Orchestrator

Overview

Welcome to the AI Engineer assessment! In this assignment, you will build an end-to-end cloud-native system for a **Multi-Tenant WhatsApp AI Support & Sales Agent SaaS**.

Your system will allow multiple companies (tenants) to manage customer queries interactively over WhatsApp. It must leverage **LangGraph** to process incoming text and media inquiries, send rich responses (text, images, and documents), toggle **WhatsApp's native typing indicators** while "thinking" to reduce user drop-offs, and store all conversations in a multi-tenant **MongoDB** (or Cloud SQL) database. The final system will include a **lightweight frontend monitoring dashboard** and be deployed to the cloud.

 **Time Expectation:** You have **48 hours** to complete this assignment. Focus on a clean, architectural prototype that highlights how state flows through your agent and how you handle async events (webhooks).

Business Scenario

Imagine a modern Customer Engagement SaaS used by different retail brands (Tenants).

- **Tenant A (Luxury Furniture Store):** Sells luxury items. Customers often ask for "product catalogs" (PDFs) or "showroom images" (PNGs).
- **Tenant B (Automotive Care):** Schedules service appointments and sends "invoice sheets" (PDFs) or "repair diagrams" (JPGs).

Your system must handle incoming customer messages, dynamically retrieve the brand's instruction context, determine whether the customer is requesting catalogs/images, trigger a "typing indicator" to keep the user engaged during LLM latency, construct and send the appropriate text and media replies via the WhatsApp Cloud API, and log the state dynamically for a monitoring dashboard.

Core Requirements & Tasks

Task 1: Multi-Tenant Database Design

Design a database schema using **MongoDB** (or PostgreSQL/Cloud SQL) supporting multiple tenants:

- **Tenant (Company):** Unique identifier, name, prompt directions (system instructions), and a **Media Library** (pre-seeded URLs mapping query terms to assets, e.g., "catalog" -> "https://example.com/catalog.pdf", "sofa" -> "https://example.com/sofa.jpg").

- **Customer Interaction (Chat Session):** Customer phone number, associated Tenant ID, custom session status (WAITING_FOR_BOT, AGENT_RESPONDING, RESOLVED), and context variables.
- **Message Audit Log:** Stores inbound/outbound records including timestamp, sender, text content, and media attachments (URLs/MimeTypes).

Task 2: WhatsApp Cloud API Integration

Set up a Meta Developer account and configure a WhatsApp Sandbox number. Your backend should interface directly with the standard Meta Graph API endpoints:

- **Read Receipts & Typing Indicator:** Upon receiving an inbound message, immediately send a POST request to mark the message as read, followed by starting a native WhatsApp typing indicator.
 - *Tip:* Toggle the typing status using Meta's typing_indicator body:
 POST /v20.0/<PHONE_NUMBER_ID>/messages


```
{
  "messaging_product": "whatsapp",
  "recipient_type": "individual",
  "to": "<CUSTOMER_PHONE>",
  "type": "typing_indicator",
  "typing_indicator": {
    "type": "text"
  }
}
```
- **Rich Media Dispatches:** Create helper methods to send:
 - Regular Text Messages (supporting Markdown formatting like **bold** and *_italics_*).
 - Image Messages ("type": "image" containing a public URL).
 - Document Messages ("type": "document" containing a public URL and a filename).

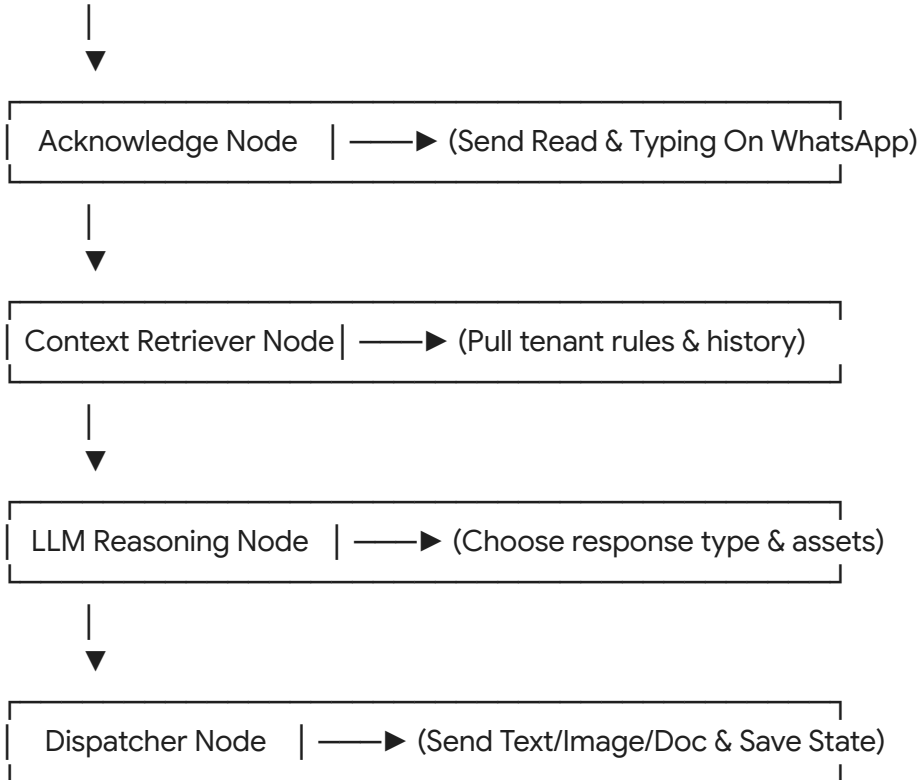
Task 3: Agentic Orchestration with LangGraph

Use **LangGraph** to model the processing pipeline for an incoming customer message. Define a stateful graph that coordinates the following nodes:

1. **Acknowledge Node:** Receives the inbound message payload. Instantly fires off the "read receipt" and "typing indicator" via your WhatsApp helper, saving the message to the database state as PENDING_RESPONSE.
2. **Context Retriever Node:** Pulls the Tenant's prompt, media catalog rules, and the last 5 messages of chat history from the database.
3. **LLM Reasoning Node:** Invokes your LLM (OpenAI/Anthropic) to determine the next conversational step.
 - *Agentic Decision-Making:* The LLM must decide if it should reply with a plain text string, or trigger a tool to attach a document (e.g. PDF Catalog) or image from the Tenant's media library because the user requested visual/data assets.

4. **Dispatcher Node:** Constructs the appropriate WhatsApp payload (text, image, or document) and sends it. It then records the outgoing response in the database, automatically extinguishing the typing indicator.

[Webhook Inbound]



Task 4: Async Webhook Handler

- Build an API server (FastAPI, Express, or NestJS).
- Expose a secure POST endpoint (e.g., /api/webhooks/whatsapp) alongside a GET verification endpoint (for Meta's Webhook verification challenge).
- **Asynchronous Execution (Critical):** Do *not* wait for the LangGraph agent to finish generating before responding to Meta. The webhook must immediately return 200 OK to Meta within 3 seconds to avoid duplicate delivery retries, kicking off the LangGraph loop in an asynchronous background thread or task worker.

Task 5: Lightweight Frontend Dashboard

Build a simple, responsive dashboard (React, Vue, or plain HTML/Tailwind CSS) for business owners to audit their agent's work:

- **Tenant Switcher:** Easily toggle between viewing Tenant A and Tenant B.
- **Live Chat Monitor:** A list of active phone numbers conversing with the bot. Selecting a number displays a stylized chat thread showing:

- User text messages.
- Bot messages with visual indicators for sent images or downloadable PDF badges.
- Metadata indicators showing when the bot was in the "typing..." state.
- **Broadcast Campaign Drawer:** An interface allowing administrators to select a cohort and trigger a predefined template message broadcast (e.g., "Send New Catalog Promo") to targeted numbers.

Task 6: Cloud Deployment (GCP Cloud Run Preferred)

Deploy the full application to a cloud hosting environment.

- **Deployment Platform:** **GCP Cloud Run is preferred**, but you are free to deploy on any platform of your choice (such as **Render**, **Fly.io**, **Railway**, etc.).
- **Containerization:** Create a single, production-ready Dockerfile that packages both backend processes and exposes ports correctly.
- **Env Configuration:** Use Secret Manager or direct environment variables for database URLs, LLM APIs, and Meta Developer tokens.
- **Webhook Mapping:** Point your Meta App's webhook configuration to the live, secure https URL assigned by your cloud hosting provider to test incoming messages in real-time.



Technical Constraints & Stack

- **Database:** MongoDB (Atlas M0 Free Tier) or Cloud SQL / managed SQL equivalents.
- **AI Orchestration:** langgraph (Python or JS/TS).
- **Messaging APIs:** Meta WhatsApp Business Cloud API.
- **Backend Framework:** Python (FastAPI/Flask) or Node.js (Express/NestJS).
- **Frontend UI:** HTML5/CSS3, Tailwind CSS, and a component framework (React, Vue, Next.js).
- **Deployment Platform:** GCP Cloud Run (Preferred), Render, Fly.io, or equivalent container platform.



Bonus Points (If you have extra time)

- **Webhook Security:** Validate the incoming webhook payloads using the X-Hub-Signature-256 header to ensure they genuinely originate from Meta.
- **Inbound Media Parsing:** If a user sends an image to the agent, use a Multimodal LLM (like GPT-4o or Gemini Flash) to parse and describe the image in the conversation state.
- **Fallback Handover:** If the LLM generates a sentiment score indicating frustration, the LangGraph flow updates the state status to NEEDS_HUMAN and halts further auto-replies, highlighting the chat in red on your frontend dashboard.



Deliverables

1. **Source Code:** A link to a clean, structured GitHub repository.
2. **Deployed URLs:** A live, working URL for both the admin dashboard and public backend webhook handler.

3. Documentation (README.md):

- Quick-start instructions for setting up environment variables (.env).
- Step-by-step instructions to run the stack locally.
- An architectural breakdown of your LangGraph schema (state representation, nodes, and edges).
- Clear documentation on the chosen deployment environment and setup details.

4. Demo Video (Loom/YouTube): A 3-5 minute video demonstrating:

- The frontend showing Tenant A's dashboard.
- Sending a message to the bot and showing the **"typing..."** status active on your phone screen while the LLM thinks.
- The bot replying with a customized message, dynamic catalog image, or PDF document.
- Switching to Tenant B and showing separate, customized answers.
- Showing how state values and message history logs change on the dashboard.



Evaluation Criteria

- **Agentic Execution & UX (25%):** Correct usage of LangGraph. Does the bot handle state, history, and media attachments smartly? Is the typing indicator implemented smoothly to reduce the perception of delay?
- **Cloud Deployment & Architecture (25%):** Secure, functional deployment (GCP Cloud Run preferred, but points awarded for any fully functional platform like Render/Fly.io). Proper handling of webhook asynchronous loops so the server isn't held up waiting for LLM completion.
- **Frontend Integrity (20%):** Clean design, functional multi-tenant toggling, and complete rendering of both conversation logs and rich media elements.
- **Meta API Mastery (20%):** Correct, robust communication with Meta's endpoints. Successful ingestion of inbound payloads.
- **Code Quality (10%):** Organized file structure, clear separation of DB interfaces, and understandable comments describing logical branches.

Best of luck! We're excited to see your multi-tenant agent come to life on WhatsApp.